

React JSX Hands-On Lab — Solution

Building the Office Space Rental Listing ("officespacerentalapp") with JSX

Objectives

- Define JSX
- Explain ECMAScript
- Explain `React.createElement()`
- Explain how to create React nodes with JSX
- Define how to render JSX to the DOM
- Explain how to use JavaScript expressions in JSX
- Explain how to use inline CSS in JSX

In this hands-on lab, you will learn how to:

- Use JSX syntax in React applications
- Use inline CSS in JSX

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

Create a React application named "officespacerentalapp" that uses JSX to create elements and attributes and to render the DOM. The page should include a heading, an image attribute for the office space photo, an object describing a single office (Name, Rent, Address), and a list of office objects rendered by looping through the data. The Rent value must be styled with inline CSS: red when it is below 60000 and green when it is 60000 or above.

Solution — Step by Step

Step 1: Create the React project "officespacerentalapp"

Open a terminal and run the following command to scaffold the project:

```
npx create-react-app officespacerentalapp
```

Once the setup finishes, navigate into the project folder:

```
cd officespacerentalapp
```

Step 2: Open the application in VS Code

```
code .
```

Step 3: Understand JSX vs. `React.createElement()`

JSX is syntactic sugar that the Babel compiler (part of the create-react-app build, which targets modern ECMAScript) transforms into `React.createElement()` calls. Writing a heading in JSX like this:

```
const heading = <h1>Office Space Rentals</h1>;
```

is equivalent to writing it directly with `React.createElement()`, without JSX:

```
const heading = React.createElement(  
  'h1',  
  null,  
  'Office Space Rentals'  
);
```

JSX is used throughout this component because it is easier to read and lets HTML-like markup and JavaScript expressions be combined directly in the render output.

Step 4: Create the heading element

In `src/App.js`, create a heading element for the page using JSX:

```
<h1>Office Space Rentals</h1>
```

Step 5: Create an office object with Name, Rent, and Address

Define a single office object with the required properties, including an image attribute:

```
const office = {  
  id: 1,  
  name: 'Skyline Business Center',  
  rent: 55000,  
  address: 'MG Road, Bengaluru',  
  image: 'https://via.placeholder.com/150'  
};
```

Use JavaScript expressions inside JSX (with curly braces) to render its properties, and use the image attribute for the photo:

```
<div className="office-card">  
  <img src={office.image} alt={office.name} width="150" />  
  <h2>{office.name}</h2>  
  <p>Address: {office.address}</p>  
  <p>Rent: ₹{office.rent}</p>  
</div>
```

Step 6: Create a list of office objects

Extend the single object into an array so multiple office spaces can be looped through and rendered:

```
const offices = [  
  {  
    id: 1,  
    name: 'Skyline Business Center',  
    rent: 55000,  
  },  
];
```

```

    address: 'MG Road, Bengaluru',
    image: 'https://via.placeholder.com/150'
  },
  {
    id: 2,
    name: 'Metro Corporate Hub',
    rent: 72000,
    address: 'Connaught Place, New Delhi',
    image: 'https://via.placeholder.com/150'
  },
  {
    id: 3,
    name: 'Riverside Workspace',
    rent: 48000,
    address: 'Marine Drive, Mumbai',
    image: 'https://via.placeholder.com/150'
  }
];

```

Step 7: Loop through the list using JSX and JavaScript expressions

Use the array's map() method inside JSX to render one card per office space:

```

<ul className="office-list">
  {offices.map(item => (
    <li key={item.id} className="office-card">
      <img src={item.image} alt={item.name} width="150" />
      <h2>{item.name}</h2>
      <p>Address: {item.address}</p>
    </li>
  ))}
</ul>

```

Step 8: Apply inline CSS to color the Rent value

Add the Rent paragraph inside the loop and use an inline style object with a JavaScript expression: red when rent is below 60000, green when it is 60000 or above:

```

<p style={{ color: item.rent < 60000 ? 'red' : 'green' }}>
  Rent: ₹{item.rent}
</p>

```

Complete App.js

Putting all the pieces together, the complete App.js file looks like this:

```

import React from 'react';
import './App.css';

function App() {

```

```

const offices = [
  {
    id: 1,
    name: 'Skyline Business Center',
    rent: 55000,
    address: 'MG Road, Bengaluru',
    image: 'https://via.placeholder.com/150'
  },
  {
    id: 2,
    name: 'Metro Corporate Hub',
    rent: 72000,
    address: 'Connaught Place, New Delhi',
    image: 'https://via.placeholder.com/150'
  },
  {
    id: 3,
    name: 'Riverside Workspace',
    rent: 48000,
    address: 'Marine Drive, Mumbai',
    image: 'https://via.placeholder.com/150'
  }
];

return (
  <div className="App">
    <h1>Office Space Rentals</h1>
    <ul className="office-list">
      {offices.map(item => (
        <li key={item.id} className="office-card">
          <img src={item.image} alt={item.name} width="150" />
          <h2>{item.name}</h2>
          <p>Address: {item.address}</p>
          <p style={{ color: item.rent < 60000 ? 'red' : 'green' }}>
            Rent: ₹{item.rent}
          </p>
        </li>
      ))}
    </ul>
  </div>
);
}

export default App;

```

Step 9: Render JSX to the DOM

In `src/index.js`, the `App` component (built from JSX) is rendered into the DOM node with id "root" in `public/index.html` — this is standard create-react-app boilerplate and normally does not need to change:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Step 10: Build and run the application

```
npm start
```

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

The page displays the "Office Space Rentals" heading followed by a card for each office space, showing its image, name, address, and rent. Rent values below 60000 appear in red, and rent values of 60000 or above appear in green, confirming that the JSX expressions, the looped list, and the inline CSS are all working correctly.

Result

A React application named "officespacerentalapp" was successfully created using JSX to define the page heading, the image attribute, an office object with Name, Rent, and Address, and a looped list of office objects rendered with `map()`. JavaScript expressions inside curly braces were used to display the object's properties, and inline CSS conditionally colored the Rent value red or green based on its value. The application was verified by running it locally and viewing it in the browser at `http://localhost:3000`.